

TRABALHO SEMESTRAL EM GRUPO

Pipeline de Integração de Dados End-to-End

Disciplina: Data Integration · Curso: Sistemas de Informação · Semestre: 2026.1

Professor	Prof. Me. Andre Insardi
Modalidade	Trabalho em grupo (3 a 5 alunos)
Peso na nota final	30% (Módulo 3)
Stack obrigatória	Python + Docker + Apache Airflow + Postgres

1. Contexto e Justificativa

Ao longo do semestre estudamos os fundamentos de integração de dados, ecossistemas de Big Data, containerização com Docker, processos de ETL em Python e orquestração de pipelines com Apache Airflow. Este trabalho consolida todos esses conhecimentos em um único projeto end-to-end, simulando um cenário corporativo real em que a equipe de Engenharia de Dados precisa entregar um pipeline operacional, documentável e reproduzível.

O exercício é deliberadamente próximo da prática de mercado: vocês escolherão uma fonte de dados real, definirão uma arquitetura, implementarão o pipeline com as ferramentas vistas em aula, validarão a qualidade dos dados e apresentarão os resultados como uma entrega técnica formal.

3. Escopo do Trabalho

3.1. Visão Geral

Cada grupo deverá construir um pipeline completo que extraia dados de pelo menos duas fontes heterogêneas, transforme-os com regras de negócio definidas pelo grupo, carregue-os em um banco de dados analítico (Postgres) e disponibilize ao menos três consultas/visualizações de valor para um suposto usuário final.

3.2. Requisitos Obrigatórios (must-have)

1. Pelo menos 2 fontes de dados heterogêneas (ex.: CSV + API REST; CSV + JSON; Banco SQL + arquivo XML; API + scraping autorizado).
2. Pelo menos 1 fonte semi-estruturada ou não estruturada (JSON, XML, logs ou similar).
3. Pipeline ETL desenvolvido em Python, com código organizado em módulos (extract.py, transform.py, load.py ou estrutura equivalente).
4. Ambiente containerizado com Docker (Dockerfile + docker-compose.yml para subir Postgres e Airflow).
5. Orquestração com Apache Airflow: DAG funcional, agendada, com pelo menos 4 tasks e tratamento de dependências.

6. Persistência em Postgres com modelagem dimensional simples (1 fato + ao menos 2 dimensões) ou modelo relacional justificado.
7. Validações de qualidade de dados (mínimo 3 regras: nulos, duplicidades, ranges, integridade referencial, etc.).
8. Repositório Git público (GitHub) com histórico de commits distribuído entre os membros.
9. Diagrama de arquitetura do pipeline (Mermaid, draw.io, Lucidchart ou similar).

3.3. Requisitos Desejáveis (nice-to-have, contam para nota extra)

- Testes automatizados (pytest) cobrindo as funções de transformação.
- Camada de logs estruturados (JSON) e monitoramento via Airflow.
- Dashboard simples (Metabase, Streamlit ou Power BI) consumindo o Postgres.
- Tratamento de incrementalidade (carga incremental ao invés de full-refresh).
- Uso de variáveis de ambiente / .env para credenciais (sem hardcoded).
- CI/CD básico (GitHub Actions rodando lint e testes).

4. Sugestões de Fontes de Dados

O grupo é livre para escolher o domínio do projeto. Abaixo, fontes públicas e gratuitas como inspiração:

4.1. Dados Governamentais e Públicos

- Portal de Dados Abertos do Governo Federal (dados.gov.br)
- IBGE (sidra.ibge.gov.br) — estatísticas demográficas e econômicas
- Banco Central do Brasil (api.bcb.gov.br) — câmbio, juros, inflação
- Receita Federal — base de CNPJ pública
- Portal da Transparência — gastos públicos

4.2. APIs Abertas

- Spotify API — músicas, artistas, playlists
- OpenWeather API — dados meteorológicos
- GitHub API — repositórios, commits, issues
- TMDb API — filmes e séries
- CoinGecko API — cripto e mercado financeiro

4.3. Datasets Estruturados

- Kaggle Datasets (kaggle.com/datasets)
- UCI Machine Learning Repository
- Brasil.io — dados públicos brasileiros estruturados

Recomenda-se que o tema escolhido tenha relevância de negócio (não apenas técnica) — algo que vocês conseguiriam apresentar para um stakeholder não-técnico.

5. Stack Técnica Obrigatória

Camada	Ferramenta	Versão mínima
Linguagem	Python	3.10+
Containerização	Docker + Docker Compose	Docker 24+
Orquestração	Apache Airflow	2.8+
Banco de Dados	PostgreSQL	15+
Bibliotecas Python	pandas, requests, sqlalchemy, psycpg2, pytest (livre)	—
Versionamento	Git + GitHub (repositório público)	—

Importante: esta é a mesma stack utilizada nas aulas práticas (Aulas 03-04 — ETL Docker; Aulas 05-06 — Apache Airflow).

7. Entregáveis

A entrega será feita no Canvas (assignment correspondente) e deverá conter:

7.1. Repositório GitHub (link público)

- Código-fonte completo, organizado em pastas (/dags, /etl, /sql, /docs, /tests).
- README.md com: descrição do projeto, instruções de execução passo-a-passo, diagrama de arquitetura, descrição das fontes.
- Dockerfile e docker-compose.yml funcionais (correção testará com docker compose up).
- Arquivo requirements.txt com dependências Python.
- Histórico de commits distribuído entre os membros (não serão aceitos repositórios com 95% dos commits em um único autor).

7.2. Relatório Técnico (PDF, máximo 12 páginas)

- Capa com identificação do grupo, título do projeto e título da disciplina.
- Sumário executivo (1 página).
- Descrição do problema e justificativa do tema.
- Arquitetura da solução (com diagrama).
- Detalhamento das fontes de dados e regras de transformação.
- Modelagem do banco de dados (DER ou modelo dimensional).

- Descrição da DAG do Airflow.
- Validações de qualidade implementadas.
- Resultados (3 consultas de valor + screenshots).
- Limitações e possíveis evoluções.
- Referências bibliográficas (ABNT).

7.3. Apresentação (PDF do slide deck)

- Máximo 12 slides.
- Duração: 15 minutos + 5 minutos de Q&A.
- Demonstração ao vivo do pipeline rodando (mínimo 2 minutos).
- Todos os membros do grupo devem falar.

8. Critérios de Avaliação (Rubrica)

A nota final do trabalho será composta por 5 dimensões, cada uma avaliada de 0 a 10:

Dimensão	O que será avaliado	Peso
1. Arquitetura e Decisões Técnicas	Coerência da arquitetura, justificativa das escolhas, modelagem do banco, separação de responsabilidades no código.	20%
2. Implementação do ETL	Qualidade do código Python, aderência aos requisitos obrigatórios, uso correto de Docker, robustez do pipeline.	25%
3. Orquestração com Airflow	DAG funcional, dependências corretas, tratamento de falhas, agendamento, uso adequado de operadores.	20%
4. Qualidade de Dados e Reprodutibilidade	Validações implementadas, logs, capacidade de o pipeline rodar do zero em outra máquina sem ajustes.	15%
5. Documentação e Apresentação	Clareza do README, qualidade do relatório, organização dos slides, domínio na apresentação oral.	20%
Total		100%

Bonificação (até +1,0 ponto extra)

Grupos que entregarem itens "nice-to-have" (testes automatizados, dashboard, CI/CD, carga incremental) podem somar até 1,0 ponto extra na nota final do trabalho.

9. Avaliação Individual e Free-rider

Embora o trabalho seja em grupo, a nota é individual. Critérios:

- Cada membro deve ter contribuições visíveis no histórico de commits do GitHub.
- Na apresentação, todos devem falar e responder perguntas sobre o trabalho como um todo.
- Ao final, cada aluno preencherá uma autoavaliação e avaliará anonimamente os colegas (peer review). Discrepâncias significativas geram redistribuição de nota.
- Membros sem commits ou que não demonstrem domínio do trabalho na apresentação terão nota individual reduzida.

10. Política de Uso de Inteligência Artificial

O uso de assistentes de IA (ChatGPT, Claude, Copilot, Gemini, etc.) é permitido e até incentivado, com as seguintes regras:

10. Toda contribuição de IA deve ser revisada e compreendida pelo grupo — vocês são responsáveis pelo código entregue.
11. O relatório deve conter uma seção curta declarando quais ferramentas de IA foram usadas e em quais etapas (ex.: “usamos Copilot para sugerir SQL e Claude para revisar a arquitetura”).

12. Trechos copiados sem entendimento serão penalizados na apresentação oral, quando o aluno for questionado e não souber explicar o código.
13. Plagiar repositórios prontos da internet (sem adaptação substancial) implica nota zero.

11. Como Entregar

14. Submeter no assignment “Trabalho Semestral em Grupo” no Canvas até 24/05/2026 às 23h59.
15. Anexar: (a) PDF do relatório técnico, (b) PDF do slide deck, (c) link público do GitHub.
16. Apenas um membro do grupo precisa submeter, mas o nome de todos os integrantes deve estar na primeira página do relatório.
17. Atrasos: -1,0 ponto por dia corrido até 48h. Após 48h, nota zero.

12. Canais de Dúvida

- Dúvidas técnicas: durante as aulas das Semanas 14-16 (encontros práticos supervisionados).
- Dúvidas de escopo: e-mail institucional do professor.
- Recomenda-se que os grupos validem o escopo do projeto com o professor até o final da Semana 14 (08/05/2026).

Bom trabalho!

Prof. Me. Andre Insardi · Data Integration · ESPM 2026.1